

maximizing the argument of Φ : i.e. increasing the ratio of $\text{MMD}^2(P, Q)$ to $\sqrt{V_m(P, Q)}$, and reducing the ratio of c_α to $m\sqrt{V_m(P, Q)}$.

For a given kernel k , V_m is $O(m^{-1})$, while both c_α and MMD^2 are constants. Thus the first term is $O(\sqrt{m})$, and the second is $O(1/\sqrt{m})$. Two situations therefore arise: when m is small relative to the difference in P and Q (i.e., we are close to the null), both terms need to be taken into account to maximize test power. Here, we propose to maximize (4) using the efficient computation of $\hat{c}_\alpha$ in Section 3. As m grows, however, we can asymptotically maximize the power of the test by choosing a kernel k that maximizes the t -statistic $t_k(P, Q) := \text{MMD}_k^2(P, Q) / \sqrt{V_m^{(k)}(P, Q)}$. In practice, we maximize an estimator of $t_k(P, Q)$ given by $\hat{t}_k(X, Y) := \widehat{\text{MMD}}_U^2(X, Y) / \sqrt{\hat{V}_m(X, Y)}$, with $\hat{V}_m(X, Y)$ discussed shortly.

To maintain the validity of the hypothesis test, we will need to divide the observed data X and Y into a “training sample,” used to choose the kernel, and a “testing sample,” used to perform the final hypothesis test with the learned kernel.

We next consider families of kernels over which to optimize. The most common kernels used for MMD tests are standard kernels from the literature, e.g. the Gaussian RBF, Matérn, or Laplacian kernels. It is the case, however, that for any function $z : \mathcal{X}_1 \rightarrow \mathcal{X}_2$ and any kernel $\kappa : \mathcal{X}_2 \times \mathcal{X}_2 \rightarrow \mathbb{R}$, the composition $\kappa \circ z$ is also a kernel on $\mathcal{X}_1$.³ We can thus choose a function z to extract meaningful features of the inputs, and use a standard kernel κ to compare those features. We can select such a function z (as well as κ) by performing kernel selection on the family of kernels $\kappa \circ z$. To do so, we merely need to maximize $\hat{t}_{\kappa \circ z}(X, Y)$ through standard optimization techniques based on the gradient of $\hat{t}_{\kappa \circ z}$ with respect to the parameterizations of z and κ .

We now give an expression for an empirical estimate $\hat{V}_m$ of the variance $V_m(P, Q)$ that appears in our test power. This estimate is similar to that given by Bounliphone et al. (2016, Appendix A.1), but incorporates second-order terms and corrects some small sources of bias. Though the expression is somewhat unwieldy, it is defined by various sums of the kernel matrices and is differentiable with respect to the kernel k .

$V_m(P, Q)$ is given in terms of expectations of k under P and Q in Appendix A. We replace these expectations with finite-sample averages, giving us the required estimator. Define matrices K_{XY} , $\tilde{K}_{XX}$, and $\tilde{K}_{YY}$ by $(K_{XY})_{i,j} = k(X_i, Y_j)$, $(\tilde{K}_{XX})_{ii} = 0$, $(\tilde{K}_{XX})_{ij} = k(X_i, X_j)$ for $i \neq j$, and $\tilde{K}_{YY}$ similarly to $\tilde{K}_{XX}$. Let e be an m -vector of ones, and use the falling factorial notation $(m)_r := m(m-1) \cdots (m-r+1)$. Then an unbiased estimator for $V_m(P, Q)$ is:

$$\begin{aligned} \hat{V}_m := & \frac{4}{(m)_4} \left[\|\tilde{K}_{XX}e\|^2 + \|\tilde{K}_{YY}e\|^2 \right] + \frac{4(m^2 - m - 1)}{m^3(m-1)^2} \left[\|K_{XY}e\|^2 + \|K_{XY}^\top e\|^2 \right] \\ & - \frac{8}{m^2(m^2 - 3m + 2)} \left[e^\top \tilde{K}_{XX} K_{XY} e + e^\top \tilde{K}_{YY} K_{XY}^\top e \right] \\ & + \frac{8}{m^2(m)_3} \left[\left(e^\top \tilde{K}_{XX} e + e^\top \tilde{K}_{YY} e \right) \left(e^\top K_{XY} e \right) \right] \\ & - \frac{2(2m-3)}{(m)_2(m)_4} \left[\left(e^\top \tilde{K}_{XX} e \right)^2 + \left(e^\top \tilde{K}_{YY} e \right)^2 \right] - \frac{4(2m-3)}{m^3(m-1)^3} \left[\left(e^\top K_{XY} e \right)^2 \right] \\ & - \frac{2}{m(m^3 - 6m^2 + 11m - 6)} \left[\|\tilde{K}_{XX}\|_F^2 + \|\tilde{K}_{YY}\|_F^2 \right] + \frac{4(m-2)}{m^2(m-1)^3} \|K_{XY}\|_F^2. \end{aligned} \quad (5)$$

2.2 OTHER APPROACHES TO MMD KERNEL SELECTION

The most common practice in performing two-sample tests with MMD is to use a Gaussian RBF kernel, with bandwidth set to the median pairwise distance among the joint data. This heuristic often works well, but fails when the scale on which P and Q vary differs from the scale of their overall variation (as in the synthetic experiment of Section 4). Ramdas et al. (2015a;b) study the power of

³If z is injective and κ characteristic, then $\kappa \circ z$ is characteristic. Whether any fixed $\kappa \circ z$ is consistent, however, is less relevant than the power of the $\kappa \circ z$ we choose — which is what we maximize.

the median heuristic in high-dimensional problems, and justify its use for the case where the means of P and Q differ.

An early heuristic for improving test power was to simply maximize $\widehat{\text{MMD}}_U^2$. Sriperumbudur et al. (2009) proved that, for certain classes of kernels, this yields a consistent test. As further shown by Sriperumbudur et al., however, maximizing MMD amounts to minimizing training *classification error* under linear loss. Comparing with (4), this is plainly not an optimal approach for maximizing *test* power, since variance is ignored.⁴ One can also consider maximizing criteria based on cross validation (Sugiyama et al., 2011; Gretton et al., 2012b; Strathmann, 2012). This approach is not differentiable, and thus difficult to maximize among more than a fixed set of candidate kernels. Moreover, where this cross-validation is used to maximize the MMD on a validation set (as in Sugiyama et al., 2011), it again amounts to maximizing classification performance rather than test performance, and is suboptimal in the latter setting (Gretton et al., 2012b, Figure 1). Finally, Gretton et al. (2012b) previously studied direct optimization of the power of an MMD test for a streaming estimator of the MMD, for which optimizing the ratio of the empirical statistic to its variance also optimizes test power. This streaming estimator uses data very inefficiently, however, often requiring m^2 samples to achieve power comparable to tests based on $\widehat{\text{MMD}}_U^2$ with m samples (Ramdas et al., 2015a).

3 EFFICIENT IMPLEMENTATION OF PERMUTATION TESTS FOR $\widehat{\text{MMD}}_U^2$

Practical implementations of tests based on $\widehat{\text{MMD}}_U^2$ require efficient estimates of the test threshold $\hat{c}_\alpha$. There are two known test threshold estimates that lead to a consistent test: the permutation test mentioned above, and a more sophisticated null distribution estimate based on approximating the eigenspectrum of the kernel, previously reported to be faster than the permutation test (Gretton et al., 2009). In fact, the relatively slow reported performance of the permutation approach was due to the naive Matlab implementation of the permutation test in the code accompanying Gretton et al. (2012a), which creates a new copy of the kernel matrix for every permutation. We show here that, by careful design, permutation thresholds can be computed substantially faster – even when compared to parallelized state-of-the-art spectral solvers (not used by Gretton et al.).

First, we observe that we can avoid copying the kernel matrix simply by generating permutation indices for each null sample and accessing the precomputed kernel matrix in permuted order. In practice, however, this does not give much performance gain due to the random nature of memory-access which conflicts with how modern CPUs implement caching. Second, if we rather maintain an inverse map of the permutation indices, we can easily traverse the matrix in a sequential fashion. This approach exploits the hardware prefetchers and reduces the number of CPU cache misses from almost 100% to less than 10%. Furthermore, the sequential access pattern of the kernel matrix enables us to invoke multiple threads for computing the null samples, each traversing the matrix sequentially, without compromising the locality of reference in the CPU cache.

We consider an example problem of computing the test using 200 null distribution samples on $m = 2000$ two-dimensional samples, comparing a Gaussian to a Laplace distribution with matched moments. We compare our optimized permutation test against a spectral test using the highly-optimized (and proprietary) state-of-the-art spectral solver of Intel’s MKL library (Intel, 2003–17). All results are averaged over 30 runs; the variance across runs was negligible.

Figure 1 (left) shows the obtained speedups as the number of computing threads grow for $m = 2000$. Our implementation is not only faster on a single thread, but also saturates more slowly as the number of threads increases. Figure 1 (right) shows timings for increasing problem sizes (i.e. m) when using all available system threads (here 24). For larger problems, our permutation implementation (scaling as $\mathcal{O}(m^2)$) is an order of magnitude faster than the spectral test (scaling as $\mathcal{O}(m^3)$). For smaller

⁴With regards to classification vs testing: there has been initial work by Ramdas et al. (2016), who study the simplest setting of the two multivariate Gaussians with known covariance matrices. Here, one can use linear classifiers, and the two sample test boils down to testing for differences in means. In this setting, when the classifier is chosen to be Fisher’s LDA, then using the classifier accuracy on held-out data as a test statistic turns out to be minimax optimal in “rate” (dependence on dimensionality and sample size) but not in constants, meaning that there do exist tests which achieve the same power with fewer samples. The result has been proved only for this statistic and setting, however, and generalization to other statistics and settings is an open question.

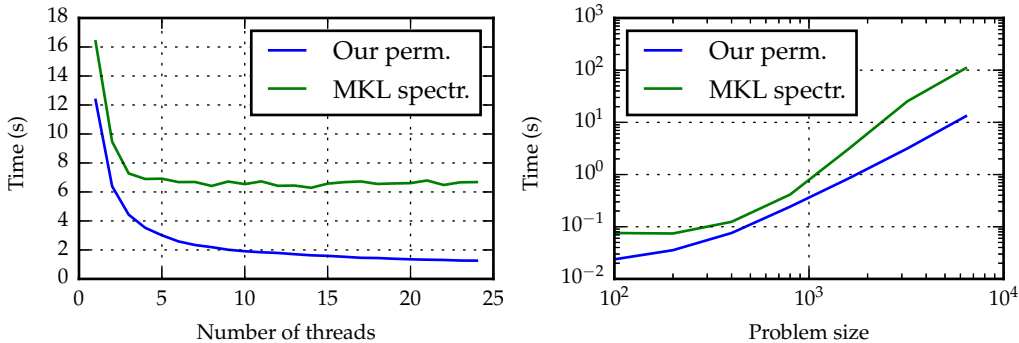


Figure 1: Runtime comparison for sampling the null distribution. We compare our optimized permutation approach to the spectral method using Intel’s MKL spectral solver. Time spent precomputing the kernel matrix is not included. *Left:* Increasing number of threads for fixed problem size $m = 2000$. Single-threaded times of other implementations: Matlab reference spectral 381s, Python permutation 182s, Shogun spectral (eigen3) 87s. *Right:* Increasing problem sizes using the maximum number of 24 system threads.

problems (for which [Gretton et al.](#) suggested the spectral test), there is still a significant performance increase.

For further reference, we also report timings of available non-parallelized implementations for $m = 2000$, compared to our version’s 12s in Figure 1 (left): 87s for an open-sourced spectral test in Shogun using eigen3 ([Sonnenburg et al., 2016](#); [Guennebaud et al., 2010](#)), 381s for the reference Matlab spectral implementation ([Gretton et al., 2012a](#)), and 182s for a naive Python permutation test that partly avoids copying via masking. (All of these times also exclude kernel computation.)

4 EXPERIMENTS

Code for these experiments is available at github.com/djsutherland/opt-mmd.

Synthetic data We consider the problem of bandwidth selection for Gaussian RBF kernels on the Blobs dataset of [Gretton et al. \(2012b\)](#). P here is a 5×5 grid of two-dimensional standard normals, with spacing 10 between the centers. Q is laid out identically, but with covariance $\frac{\varepsilon-1}{\varepsilon+1}$ between the coordinates (so that the ratio of eigenvalues in the variance is ε .) Figure 2a shows two samples from X and Y with $\varepsilon = 6$. Note that when $\varepsilon = 1$, $P = Q$.

For $\varepsilon \in \{1, 2, 4, 6, 8, 10\}$, we take $m = 500$ samples from each distribution and compute $\widehat{\text{MMD}}_U^2(X, Y)$, $\widehat{V}_m(X, Y)$, and $\widehat{c}_{0.1}$ using 1 000 permutations, for Gaussian RBF kernels with each of 30 bandwidths. We repeat this process 100 times. Figure 2b shows that the median heuristic always chooses too large a bandwidth. When maximizing MMD alone, we see a bimodal distribution of bandwidths, with a significant number of samples falling into the region with low test power. The variance of $\widehat{\text{MMD}}_U^2$ is much higher in this region, however, hence optimizing the ratio $\hat{t}$ never returns these bandwidths. Figure 2c shows that maximizing $\hat{t}$ outperforms maximizing the MMD across a variety of problem parameters, and performs near-optimally.

Model criticism As an example of a real-world two-sample testing problem, we will consider distinguishing the output of a generative model from the reference distribution it attempts to reproduce. We will use the semi-supervised GAN model of [Salimans et al. \(2016\)](#), trained on the MNIST dataset of handwritten images.⁵ True samples from the dataset are shown in Figure 3a; samples from the learned model are in Figure 3b. [Salimans et al. \(2016\)](#) called their results “completely indistinguishable from dataset images,” and reported that annotators on Mechanical Turk were able to distinguish samples

⁵We used their code for a minibatch discrimination GAN, with 1000 labels, and chose the best of several runs.